DTU 02201 Agile Hardware Design:
Formal Verification & Hardware Compilation

Amelia Dobis (she/her) – PhD Candidate in Computer Science, Princeton University
(amelia.dobis@princeton.edu, cs.princeton.edu/~ad4048)
10th November 2026

These notes contain spoilers, i.e. answers to the questions. Please refer to the spoiler-free version of the notes if you want to be able to answer the questions on your own (recommended).

Part 0: Setup

The tools presented in this lecture are very recent reintroductions to chisel, therefore please make sure that you are using a Chisel version > 7.16.0. This can be set in your `build.sbt` file. Feel free to use the following `build.sbt`:

```
ThisBuild / scalaVersion     := "2.13.18"
ThisBuild / version          := "0.1.0"
ThisBuild / organization     := "dtu.ese"

val chiselVersion = "7.16.0"

lazy val root = (project in file("."))
  .settings(
    name := "timer",
    libraryDependencies ++= Seq(
      "org.chipsalliance" %% "chisel" % chiselVersion,
      "org.scalatest" %% "scalatest" % "3.2.19" % "test",
    ),
    scalacOptions ++= Seq(
      "-language:reflectiveCalls",
      "-deprecation",
      "-feature",
      "-Xcheckinit",
      "-Ymacro-annotations",
    ),
    addCompilerPlugin("org.chipsalliance" % "chisel-plugin" % chiselVersion cross
CrossVersion.full),
  )
```

The lecture notes begin on the next page.

Part I: Introduction to Formal Verification in Chisel

In this lecture we're going to cover a different, in some cases more efficient and more powerful, method for verifying the correctness of a hardware design.

Let's start with a design that will be our running example:

```
/// Timer circuit generator, that counts up to a given parameter n
/// When n is reached, the counter emits a done signal that remains
/// high so long as the circuit isn't reset.
/// @param n: Int . 0 < n < 2^32
class Timer(n : Int) extends Module {
    val io = IO(new Bundle{
        val done = Output(Bool())
    })
    // sanity check
    require(n > 0)

    // compute the smallest width to store n
    val width: Int = log2Ceil(n)
    val count = RegInit(0.U(width.W))

    // increment counter every cycle
    count := count + 1.U

    // emit done when count surpasses n
    io.done := (count > (n - 1).U)
}
```

Question: There's a bug in this design, can anyone spot it?

Answer: The design will never raise the done signal when n is a power of 2, as the count will overflow.

Question: What method would you typically use to catch such a bug?

Answer: Multiple 'typical' methods exist, e.g. writing out a unit-test style testbench which checks for edge cases the user thought of, one can also simply randomly stimulate the inputs in hopes that they will catch the error, etc...

Generally speaking you would want to have some form of expected behavior encoded somewhere. This is typically referred to as a *specification*, which can be encoded as an *assertion* either directly alongside the design, or in a *test harness* like a testbench. If the assertion only relates to the inputs and outputs of our design, then we can safely have it exist solely in a testbench. Otherwise, if the assertion relates to internal state, such as our `count` register, then we have to either expose the register as a port or embed our assertion in the design (the latter is more typical).

Question: What would the expected behavior of this design look like (as an assertion in Chisel using `AssertProperty(...)`)?

Answer:

```
val pastCount = RegNext(count) // store the value of count from the previous cycle
AssertProperty(
  pastCount < count && // count is monotonically increasing
  (count === n) -> io.done // done signal gets triggered properly
)
```

→ The question now is: what do we do with this specification once we have it. In our case, there several different ways we can use our specification to check for bugs in our design.

1. **Simulation:** We could write a testbench instantiating our design and design manually. The issue with this approach, as with all *dynamic verification* approaches, is:
   - *How do we pick the correct inputs to test?*
   - *How do we know if we covered all of the cases needed to expose any bugs?*
2. **Fuzzing or Exhaustive Testing:** In theory, we could avoid this problem by simply randomly throwing inputs at the design and using the feedback (coverage information, whether we found a bug, etc.) to guide how we pick our next inputs, which is known as *fuzzing*. Otherwise, we can just try **all** of the possible inputs, known as *exhaustive testing*. These two methods are still *dynamic*, i.e. they rely on fully simulating a design in order to achieve a result. This expends considerable resource when your input space is large and therefore it could take several hours to fully explore the input space of a 32bit ALU.
3. **Formal Verification:** In formal verification, the goal is to 'prove' beyond a doubt that the design matches its specification and automatically find the case where it doesn't when necessary.

In Chisel, "formal tests" depend entirely on the assertions inside the module we are verifying. Once the you have these assertions, you can formally verify the module simply by using `verifyRes(...)`.

```
import chiseltest.Formal._
// ... other chisel imports

object TimerFormalTest extends App with Formal {
  verifyRes(new Timer(8), /* backend parameters */) match {
    case Sat(witness) => println(s"Found a counter example for n = 8")
    case Unsat        => println(s"Design matches specification for n = 8")
  }
}
```

Chisel has 2 formal testing functions in the `Trait chiseltest.Formal`:

- `verify(<module>, <backend_parameters>)`: which formally verifies a given module using the requested backend and directly prints any counterexample that was found.
- `verifyRes(<module>, <backend_parameters>)`: which does the same as `verify` but returns the result as an abstract data type `BMCResult` we can pattern match on (like in the example above).

Verifying Generators:
Notice that we only verified a single instance of our generator. That's because generator paraments, e.g. `n: Int` in our Timer, exist on a different *layer* than hardware signals. This means that by the time the compiler is converting the design into a form that can be verified, the parameter is already converted into a constant, i.e. the compiler never sees it!

To understand what's going on in this magic `verify` function, we need to first understand a little bit about what's happening under the hood during the compilation of the design into something that can be verified.

Part II: Under the Hood of the Chisel Compiler

Let's take a look at the chisel compilation pipeline, i.e. what happens to a Chisel design for to be transformed into Verilog or, in our case, BTOR2 (more on that later).

The Chisel Compilation Pipeline
Given a scala file describing a Chisel design, the following stages occur:

1. Elaboration: Chisel generator get concretized (given real values), i.e. the "scala" code is evaluated to be left with only "real" chisel code. *In our timer example `n: Int` is a generator parameter, since it uses a scala type `Int`. After evaluation, all paraments get assigned concrete values, e.g. `n = 8`.*

2. Lower to CHIRRTL: The last stage of our "frontend", i.e. the part that is handled by scala, is to convert our design into a format that CIRCT (the underlying unified hardware compiler) can parse. For historical reasons, Chisel has its own custom Intermediate Representation (IR) called FIRRTL (Flexible IR for Register Transfer Level (RTL)). FIRRTL has several abstraction levels, the only one that is currently used in its own textual format is CHIRRTL, the form of FIRRTL that is the closest to Chisel. Here is our Timer design converted to CHIRRTL with `n = 7`.

```
circuit Timer :
  public module Timer :
    input clock : Clock
    input reset : UInt<1>
    output io : { done : UInt<1> }

    regreset count : UInt<3>, clock, reset, UInt<3>(0h0)
    connect count, tail(add(count, UInt<1>(0h1)), 1)
    connect io.done, gt(count, UInt<3>(0h6))
```

Notice that we are passed elaboration at this point, so all of high-level functional programming concepts, and generator parameters have been elaborated and concretized. So here, instead of seeing the operations on `n` we had in our source code, we instead only see the result of these operations, e.g. `io.done := (count > n-1)` becomes `connect io.done, gt(count, 6)`. Some procedural constructs, like `when` blocks are still kept at this stage.

3. Lower to LowFIRRTL: After CHIRRTL gets parsed and consumed by the compiler (CIRCT), it is immediately converted to a structural hardware. This form is still technically FIRRTL, just low-level, where things like `when` get expanded into a series of multiplexers, etc... Additionally, since this is now in the internals of CIRCT, this format is in MLIR (LLVM-based IR format for domain-specific compilers).

```
module {
  firrtl.circuit "Timer" {
    firrtl.module @Timer(
      in %clock: !firrtl.clock, in %reset: !firrtl.uint<1>,
      out %io_done: !firrtl.uint<1>
    ) {
      %c6_ui3 = firrtl.constant 6 : !firrtl.uint<3>
      %c1_ui1 = firrtl.constant 1 : !firrtl.uint<1>
      %c0_ui3 = firrtl.constant 0 : !firrtl.uint<3>

      %io_done_0 = firrtl.wire {name = "io_done"} : !firrtl.uint<1>
      firrtl.matchingconnect %io_done, %io_done_0 : !firrtl.uint<1>

      %count = firrtl.regreset %clock, %reset, %c0_ui3 : !firrtl.uint<3>
      %_count_T = firrtl.add %count, %c1_ui1 : !firrtl.uint<4>
      %_count_T_1 = firrtl.bits %_count_T 2 to 0 : !firrtl.uint<3>
      firrtl.matchingconnect %count, %_count_T_1 : !firrtl.uint<3>

      %_io_done_T = firrtl.gt %count, %c6_ui3 : !firrtl.uint<1>
      firrtl.matchingconnect %io_done_0, %_io_done_T : !firrtl.uint<1>
    }
  }
}
```

LowFIRRTL is in Single Static Assignment (SSA) form, so all nested operations are split into their own line with their own SSA name (the names that start with `%`). This form of encoding an a grouping of operations in a IR, like FIRRTL, in MLIR is called a 'dialect'. LowFIRRTL is thus also referred to as "the `firrtl` dialect". MLIR operations are typically in the form:

```
%<ssa_name> = <dialect>.<operation> <operands...> : !<types>
```

Question: What has changed between the Timer's CHIRRTL and LowFIRRTL forms?

Answer: The literals have been hoisted into separate constant declarations, an explicit wire has been generated for the done output connection, and the bit cropping done by the addition (omitting the last carry bit) is made explicit.

4. Converting FIRRTL into the Core Dialects: CIRCT is a unified compiler, this means that it not only supports Chisel as its frontend, but also other hardware languages such as Kanagawa or Verilog. To do so, it defines a "core" IR that represents a generic form of RTL hardware. This IR is made up of 3-5 dialects (3 design + 2 verification) which are jointly called the "core dialects". As with every frontend, we must eventually lower our source design into these core dialects in order to use CIRCT's optimizations and backends. That is thus the next step in our pipeline, where we convert the `firrtl` dialect into these core dialects, by matching all `firrtl` specific operations to equivalent core dialect ones, while adapting the abstractions where needed.

```
module {
  hw.module @Timer(in %clock : !seq.clock, in %reset : i1, out io_done : i1) {
    %c-1_i3 = hw.constant -1 : i3
    %c1_i3 = hw.constant 1 : i3
    %c0_i3 = hw.constant 0 : i3
    %count = seq.firreg %0 clock %clock reset sync %reset, %c0_i3 : i3
    %0 = comb.add bin %count, %c1_i3 {sv.namehint = "_count_T"} : i3
    %1 = comb.icmp bin eq %count, %c-1_i3 {sv.namehint = "io_done"} : i3
    hw.output %1 : i1
  }
}
```

Question: What has changed between the Timer's `firrtl` and core dialect forms?

Answer: Connections are no longer separate operations, rather they are done implicitly through operand passing. The bit cropping in the addition is now also done implicitly using the return type.

5. Target One of CIRCT's Backends: The final step in our pipeline is to target a specific backend from our core dialects. CIRCT implements several backends. Backends can be internal tools, e.g. for simulation (using `arcilator`), for verification using `circt-bmc`, or even for synthesis (using `circt-synth`). Backends can also simply be other formats / languages that get emitted, e.g. Verilog (for using external tools), or BTOR2 (for formal verification).

This compilation pipeline, from frontend to backend, is summarized in Figure 1.

Question: What benefits do we get from using a unified compiler (vs. a custom-built compiler)?

Answer: As frontend language developers, we get to focus on building out the specific of our language and simply target the core dialects. We then get access to many generic optimizations and tooling, such as Verilog emission, formal verification, simulation, … for free, i.e. without any additional work.

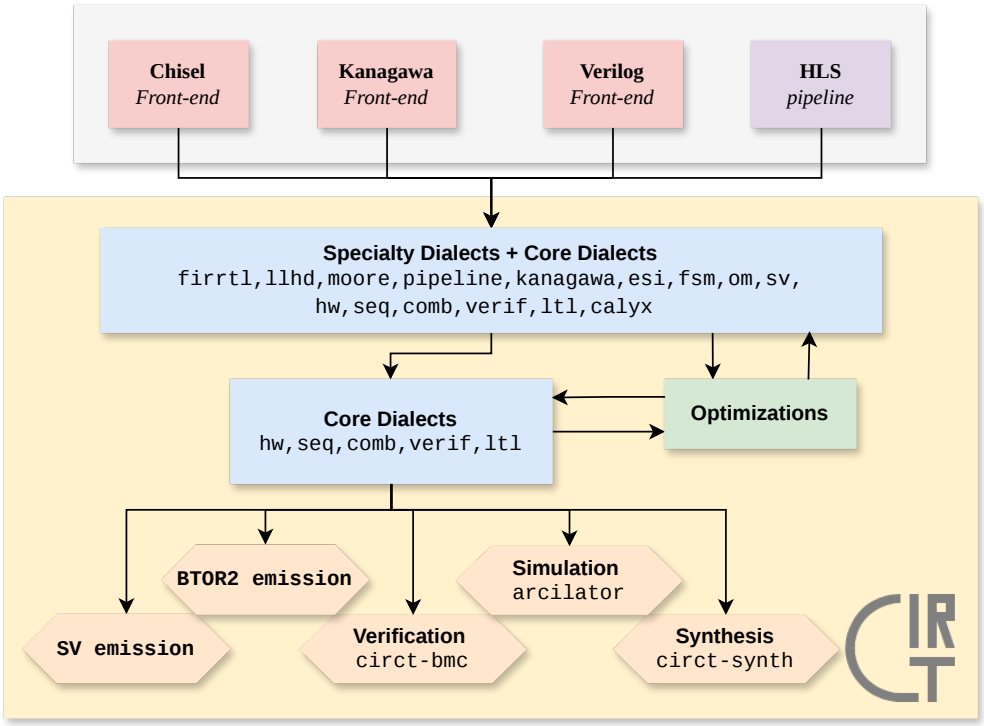


Figure 1: High-level overview of the internals of the CIRCT compiler, including the several frontend specific dialects that make up the *specialty dialects* of which *firrtl* is a part of, the core dialects, and the backends CIRCT implements.

It is important to understand the internals of Chisel's compiler when using verification backends, as the counterexamples that are found can potentially refer to signals, or even point to operations, introduced during one of these intermediate steps. Debugging without a basic understanding of the compilation pipeline is thus significantly harder.

This brings us back to our magic `verify` function from earlier. While understanding the compilation pipeline is all fine and dandy, we still have yet to explore how we are capable of "proving properties" about our design?

Part III: Under the Hood of Formal Verification

Let's say that we have a hardware design described in a generalized RTL form, i.e. the core dialects, and we want to show that it can not violate a certain set of assertions, i.e. its specification, under any possible input.

If we want to put this is more mathematical terms, we can define the following:

$$
\begin{aligned}
A &\coloneqq \text{set of assertions} \\
C &\coloneqq \text{set of constraints (assumptions)} \\
V &\coloneqq \text{set of free variables (inputs)} \\
\varphi_l^V &\coloneqq \text{a concretization of V (set of real values for inputs in V)} \\
S &\coloneqq \text{set of states where } \begin{cases} S_i \coloneqq \text{value of states in S at cycle i} \\ I(S) \coloneqq S_0 \\ T(S_i, \varphi_l) \coloneqq S_{i+1} \end{cases} \\
A_i^{S_j} &\coloneqq \text{Assertion } A_i \text{ checked on states } S_j \\
C_i^{S_j} &\coloneqq \text{Assumption } C_i \text{ checked on states } S_j
\end{aligned}
$$

With these symbols[1], we can define the problem we are trying to solve, i.e. there does not exist some concrete assignments to our inputs which violates our specification. This can also be rewritten as "all concrete assignments to our set of inputs do not violate our specification". To make things easier to prove, we use a "bounded" method, in which our proof is valid up to a cycle k. Using the terms we defined above, we want to show that:

$$
\forall \varphi_l^V . \neg \Big(I(S) \wedge \wedge_{i=0}^{k} T\big(S_i, \varphi_l^V\big) \wedge \Big(\wedge_{i=0}^{|C|} C_i^{S_k} \rightarrow \neg \wedge_{i=0}^{|A|} A_i^{S_k} \Big) \Big)
$$

If we flip this problem, we can view this as "searching for a counter-example", i.e. searching for concrete assignments to variables that make one of our assertions break. In this view, this looks very similar to a very famous problem, that of "boolean satiability" (SAT). Luckily for us, people have been building solvers for this problem for decades, known as SAT solvers. To be more specific, SAT solvers that work in a domain that is not that of boolean algebra, in our case it's bitvector logic, are known as "Satisfiability Modulo Theory" (SMT) solvers.

So, how can we use an SMT solver to solve our "bounded hardware verification[2]" problem?

Using SMT Solvers to Verify Hardware

We've covered how Chisel designs get lowered into a generic structural RTL form known as the *core dialects*. Now let's focus on how we can use the core dialect representation to generate our verification problem.

1. Create a "Model" of Our Design

The first step is to convert our design into an appropriate mathematical model that we can reason about more formally. This model is called a "Kripke Structure", a.k.a. a state transition system.

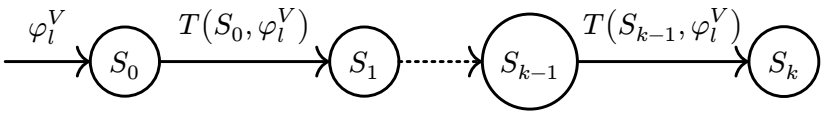


Figure 2: Abstract representation of a state transition system using our prior definitions.

Figure 2 is simply a graphical representation of the definitions we posed before. Here we can concretely see that, by modeling our design as such, we are "unrolling" its logic over k steps, where each step represents a clock-cycle for some global clock. In practice, this means that each S_i will represent the value of our stateful elements (registers, memories, etc.) at a given clock cycle i, and our transition

[1]Note here that the l in φ_l^V is simply an index used to differentiate between concretizations, as there can be several concretizations that lead to counterexamples, e.g. φ_0^V and φ_1^V can both violate the specification.

[2]This is not the official name of the problem, a more technical name will be described in a bit.

function represents the logic used to update these stateful elements, i.e. `next` logic. This representation allows us to easily check the behavior of our design a given clock cycle, as we now have a unrolled view of the system.

2. Encode our Specification

The next step is to encode our specification to match the problem we are trying to solve. In our case, assumptions become Solver constraints, which are used to reduce the input search space. For assertions however, we are not checking if our design matches the specification, as that would just yield a trivial example where it happens to be correct, rather we are looking for cases where it violates our assertions, i.e. where $\neg A_i$ holds. Therefore our assertions are encoded as "bad" states, i.e. states that should not be reachable. Our encoded specifications result in a check where given $S_j : 0 \leq j \leq k$:

$$\wedge_{i=0}^{|C|} C_i^{S_j} \rightarrow \neg \wedge_{i=0}^{|A|} A_i^{S_j}$$

In practice this encoding of the model of our design, our constraints, and our `bad` states, is done in a format called `BTOR2`.

3. Query the SMT Solver

Now that we have a model of our design and an encoding of our specification, we can create queries for the SMT solver, i.e. requests of the form "are there any assignments to some variable that make this expression hold?". In our case, at each step i, we ask the SMT Solver the following:

$$\exists \varphi_l^V : I(S) \wedge \wedge_{t=0}^{i} T\left(S_t, \varphi_l^V\right) \wedge \left(\wedge_{k=0}^{|C|} C_k^{S_i} \rightarrow \neg \wedge_{k=0}^{|A|} A_k^{S_i}\right)$$

If one of the queries at step $0 \leq j \leq k$ results in **SAT** (an assignment φ_l^V was found), then return the concrete trace $S_0, .., S_j$ for φ_l^V. This trace is commonly known as a *witness*.

If none of the queries hold, then the result is **UNSAT**.

SMT solvers use a technique known as Conflict Driven Clause Learning (CDCL) to efficiently explore the input space by using the results of previous attempts to incrementally add constraints to our queries. While a more in depth discussion about SMT solvers is outside of the scope of a single guest lecture, just remember that they are far more efficient than exhaustive testing, or even fuzzing, at exploring the entire input space.

This 3 step method, where we create a model of our design and check it until a certain bound using an SMT solver, is called *SMT-based Bounded Model Checking (BMC).*

Part IV: Creating the `verify` Function

We now know everything we need to be able to fully build the `verify` function we used in the beginning to formally verify our Timer.

Step 1: Convert our design from the core dialects to `BTOR2`. This follows our theoretical conversion where: $V_i \rightarrow$ `input`; $S \rightarrow$ `state`; $I(S) \rightarrow$ `init`; $T(S) \rightarrow$ `next`[3] $A_i \rightarrow$ `bad`; $C_i \rightarrow$ `constraint`.

Here is our running example from the earlier parts, converted into `BTOR2`:

[3]We omit φ_l^V here as we are modeling the transition for all inputs.

```
1 sort bitvec 1
2 input 1 reset
3 sort bitvec 3
4 state 3 count
5 constd 3 -1
6 constd 3 1
7 constd 3 0
8 add 3 4 6
9 eq 1 4 5
10 ite 3 2 7 8
11 next 3 4 10
```

This format is a bit particular as line numbers are used as instruction identifiers, e.g. the `1` in `2 input 1 reset` refers to the instruction in line 1 which itself is a type definition of `UInt<1>` where `UInt` is `bitvec` in BTOR2.

Slowly walking through we can see that this encodes our design as a series of `state` instructions which transition using `next` logic. If we add in a specification, e.g.

```
val pastCount = RegNext(count)
AssertProperty(pastCount < count)
AssertProperty(!(count === n.U) || io.done)
```

our BTOR2 becomes the following:

```
1 sort bitvec 1
2 input 1 reset
3 sort bitvec 3
4 state 3 count
5 state 3 pastCount
6 constd 1 0
7 state 1 hbr
8 init 1 7 6
9 constd 3 1
10 constd 3 0
11 constd 3 -1
12 add 3 4 9
13 eq 1 4 11
14 ult 1 5 4
15 constd 1 -1
16 or 1 2 7
17 xor 1 2 15
18 and 1 7 17
19 implies 1 18 14
20 not 1 19
21 bad 20
22 ite 3 2 10 12
23 next 3 4 22
24 next 3 5 4
25 next 1 7 16
```

Note in particular the addition of the `bad` instruction in line 21, which encodes the conjunction of our assertions. BTOR2 is generally best read bottom to top, so start with the `bad` instruction and work your way up.

Question: Is there anything you notice in the BTOR2 that wasn't in our original design?

Answer: There is a new `hbr` state that wasn't in the Timer. This additional state stands for "has been reset" and allows us to track if registers have been reset or not. This avoids the issue where our uninitialized garbage states might trivially violate our assertions, in that case we want to use our `reset` value as an initial value. "Has been reset" allows us to keep track of that process.

Step 2: Give our btor2 model to a bounded model checking tool using some state k. BMC tools, like `btormc`, unroll the design up until some parameter `k_max` and create the individual queries that are sent to SMT solvers like `bitwuzla`.

Step 3: Parse the result and see if the BMC tool found a counter-example!

If we put all 3 steps together, we get the following implementation of the `verify` function:

```
def verify[T <: Module](dut: => T, kmax: Int, printWitness: Boolean = false) = {
      // start by running the btor2 backend
      val btor2DUT: String = ChiselStage.emitBtor2(dut,
        firtoolOpts = Array( // Compiler flags to get a clean result
          "-disable-all-randomization",
          "-strip-debug-info",
          "-default-layer-specialization=enable"
        )
      )

      // Store the btor2 result to a temp file (using java)
      val btor2file = ...

      // Check if the given backend is in the user's path
      val executablePath: String = {...}

      /* verify the model using the given backend */
      // Setup our model checker invocation
      val command = List(
        executablePath, "--kmax=", kMax.toString, btor2file)

      // Run BMC invocation and store output
      val bmcRes = {
        val output = mutable.ArrayBuffer.empty[String]
        val exitCode: Int = command.!(ProcessLogger(output += _))
        if (exitCode != 0) throw new Exception(...)
        // store output including all newlines
        output.foldLeft("\n")((acc, o) => acc + (o + "\n")).trim
      }

      // parse result into witness
      val result = bmcRes match {
        case "" => Unsat // Note this is only true for btormc
        case s  => Sat(BasicWitness(s))
      }
      // print out result if requested
      if (printWitness) println(result.message)
      result
    }
```

This is of course a trimmed implementation to fit on one page, but all of the logic remains the same. The full implementation is available on github[4]. The goal here is to show that it is essentially only performing the 3 steps we highlighted during the lecture, with most of the heavy lifting being done in the BTOR2 Backend implemented in CIRCT.

Question: Now that we have an idea of how formal verification, particularly Bounded Model Checking, works, can you think of *one key limitation* that BMC can potentially have?

Answer: The method only provides *bounded* proofs. That means that it is always possible that, if we get UNSAT, there might exist a counter-example at a future cycle we just didn't test for (larger k).

Here's a simple example of a design that highlights the main limitation in BMC:

```
class DualCounter extends Module {
  val io = IO(new Bundle {
    val en = Input(Bool())
  })

  val prevCount = RegInit(0.U(8.W))
  val count = RegInit(1.U(8.W))

  when(io.en) {
    prevCount := count
    count := count + 1.U
  }

  // check that the register is monotonically increasing
  AssertProperty(!io.en || count > prevCount)
}
```

Similar to our previous example, this designs contains an overflow bug. This means that if we only run BMC for $k < 2^8$ then we won't catch the overflow bug, while we will always catch it for any $k > 2^8$. This highlights the main limitation in our method, i.e. its inherently bounded nature.

Question (hard): Can anyone think of a way to make this proof unbounded?

Answer: We can turn it into a proof by induction, a.k.a. *k-induction*. I would recommend looking into this method if you're interested, as it falls out of the scope of this, already long, lecture.

Wrapping up

We now know how to verify a design in Chisel. This allows us to give much stronger correctness guarantees than random testing or other dynamic techniques, as we know that, up to a certain cycle k, that there does not exist a set of inputs that violates our assertions. In the case where we do find a counter-example, we can then convert this set of inputs into a test-bench and run it in simulation to get a waveform that can make it easier to debug our design. This completely eliminates the uncertainty of explored enough inputs for our tests to be sufficient, as we will have explored all inputs.

Question (open): This brings us back to our initial question: can we modify our Timer generator from before in such a way that it allows us to verify the *generator* rather than the *generated designs* using BMC?

Answer: This is an open question in the field, as most naive approaches lead to somehow constraining the input space too much for it to fully verify the generator space.

[4] https://github.com/chipsalliance/chisel/blob/main/src/main/scala/chiseltest/formal/package.scala